Mindanao State University - Iligan Institute of Technology
College of Computer Studies
Department of Computer Applications
BCA182 Embedded Systems Programming

Last Name: _____ Date: _____
First Name: _____ Subject/Section: _____

Project 1
Smart Baby Monitoring System
100 Points

Overview

In this project, you will design and develop an Internet of Things (IoT) baby monitoring prototype using the RT-Thread RT-Spark development board. Your system should monitor selected environmental conditions, identify possible baby-crying events through audio classification, and notify a caregiver through a mobile application.

You will develop the embedded firmware for the STM32F407 microcontroller using the STM32Cube HAL framework and FreeRTOS, with Visual Studio Code and PlatformIO as the required development workflow. The board is branded RT-Thread RT-Spark, but you must not use the RT-Thread operating system or RT-Thread Studio. You must implement, integrate, and test the baby-monitoring features on the specified hardware using your own FreeRTOS-based application.

This project involves independent tasks across multiple platforms, including:

- Developing STM32Cube HAL and FreeRTOS firmware for sensor acquisition, multitasking, and local alarms
- Acquiring audio through a suitable microphone and implementing an ML-based cry/non-cry classifier
- Integrating available onboard sensors and modules into clearly justified monitoring functions
- Exchanging data securely with a backend service and storing events
- Developing a mobile application for monitoring, notifications, and event history
- Testing functional accuracy, responsiveness, reliability, and resource use

Since you are a 4th-year BS in Computer Applications, Embedded Systems Major, you are expected to integrate previously learned skills in embedded programming, device interfacing, networking, application development, and software testing. The new challenge is to make these components operate reliably as one system.

Requirements

In this project, you will be using the following hardware and software resources:

- One RT-Thread RT-Spark (Spark-1) development board, based on the STM32F407 microcontroller.
- Relevant onboard AHT10 temperature/humidity sensor, AP3216C ambient-light/proximity sensor, ICM20608 accelerometer/gyroscope, Wi-Fi module, LCD/display, LEDs, buttons, buzzer, and storage resources, subject to verification against the actual board and its schematic.
- An external microphone or audio-capture module, since microphone input is not identified as an integrated resource of the reference Spark-1 board.

- Additional external sensors or modules ONLY when the required measurement or function is not already provided by the RT-Spark board. Document the reason for each addition.
- A computer, an Android device for testing, a GitHub account, and access to a suitable local server or cloud service.

Required development resources are Visual Studio Code with the PlatformIO extension, the ST STM32 PlatformIO platform, the STM32Cube (HAL/LL) framework, a correctly integrated FreeRTOS kernel and configuration, STM32F4-compatible device drivers, Git and GitHub, and the appropriate programmer/debugger. Use Android Studio or an approved alternative to develop the mobile application. You may use STM32CubeMX to configure pins and clocks, but the submitted firmware must be reproducibly built and uploaded through PlatformIO.

IMPORTANT: The RT-Spark hardware does not require the RT-Thread operating system. Do not use RT-Thread Studio, RT-Thread OS, Arduino, ESP-IDF, or an RT-Thread-dependent driver stack in your final firmware. A board-support package written for RT-Thread is not directly compatible with FreeRTOS. Port only the necessary hardware-specific knowledge or driver logic to STM32Cube HAL, with proper attribution and verification.

Required Toolchain and Board Setup

Your firmware development workflow must follow these requirements:

- Use Visual Studio Code and PlatformIO as the primary editor, build system, upload workflow, and serial-monitor/debugging environment.
- Select the ST STM32 platform and the STM32Cube framework. In platformio.ini, the framework setting is framework = stm32cube, not framework = freertos. FreeRTOS must be included as kernel/middleware within the STM32Cube-based project.
- Use a verified STM32F407ZGT6-compatible PlatformIO board definition or prepare and document a custom RT-Spark board manifest that matches the actual MCU, clock, flash, RAM, upload method, and linker/memory settings. Do not assume that an unrelated Nucleo or Discovery board profile matches RT-Spark pins.
- Develop peripheral interfaces through STM32Cube HAL/LL and use FreeRTOS APIs for concurrency. Integrate the required FreeRTOS source, configuration, interrupt priority scheme, and timer/clock configuration into the PlatformIO project.
- Verify that the display, sensors, Wi-Fi module, and any other onboard module function under STM32Cube HAL and FreeRTOS; RT-Thread demonstration projects may be studied for hardware wiring but are not acceptable as the final application.
- Use Git and GitHub from the beginning. Commit the portable source code, board definition, configuration, tests, documentation, and wiring notes; exclude generated .pio build directories and all secrets. Explain repository branches, commits, and the tagged final version.

Before attempting the complete project, demonstrate a clean PlatformIO build, successful flashing to the RT-Spark board, serial logging, blinking an onboard LED, and at least one correctly scheduled FreeRTOS task. Include the exact build environment and a reproducible setup procedure in the GitHub README.

Tasks

The project involves five key tasks:

1. Inventory the board resources and design a complete monitoring solution.

2. Develop and validate the baby-cry detection pipeline.
3. Develop the STM32Cube HAL and FreeRTOS firmware and integrate onboard and external hardware.
4. Develop the backend service, database, and mobile application.
5. Integrate and evaluate the complete prototype.

These tasks should be tackled incrementally. Demonstrate each sensor driver, classifier, network connection, backend endpoint, and mobile screen independently before complete system integration. Use a version-controlled GitHub repository throughout development.

High Level Overview

The following is a recommended functional arrangement. Students may propose another architecture if its data flow, resource requirements, and failure handling are technically justified.

Device: RT-Spark + sensors	Processing / Cloud / Local Server	Caregiver Mobile App
Sensors + microphone -> STM32 HAL drivers + FreeRTOS tasks	Audio classification (if not feasible on-device); event processing	Current readings, device health, cry state
Wi-Fi -> MQTT or authenticated HTTP	Database + API + notification service	Alerts, history, acknowledgement
LCD, LEDs, buzzer, buttons, local log	Timestamping, aggregation, alert rules	Settings and connection status

Recommended data flow: microphones and onboard sensors acquire data; the device processes and publishes readings or compact features; a local or cloud service completes inference when necessary, stores records, and generates caregiver alerts; the mobile application displays the latest values and history. A lightweight on-device classifier is acceptable if memory and timing measurements demonstrate that it is feasible.

Onboard Hardware and Functions

Use the existing board resources before adding external hardware. Integrate the following resources as appropriate, and explain any exclusion in a component-inventory table:

Onboard resource	Required or defensible project function
AHT10 (temperature and humidity)	Measure ambient conditions near the monitoring station; display, log, and alert using documented thresholds.
AP3216C (ambient light and proximity)	Monitor room lighting; use proximity only for a local obstruction/placement indication, not as evidence of infant presence.
ICM20608 (accelerometer / gyroscope)	Detect movement or orientation change of the monitoring DEVICE; do not claim to measure infant movement or breathing.
Wi-Fi and onboard display	Send events/readings to the service; provide local status and current readings.
LEDs / RGB / matrix, buzzer, buttons	Display normal, alert, and fault states; allow a local acknowledge/mute/test action.
RTC, external flash, or TF card if enabled	Timestamp and retain essential events across temporary network interruptions.

Other onboard peripherals or modules that are present on your specific RT-Spark kit must be included in your inventory. Use those that contribute a meaningful function. Do not force irrelevant interfaces into the design simply to claim that more hardware was used. Any additional hardware must be identified in the bill of materials, with its function and the reason no equivalent onboard resource can be used.

Audio Analysis and Machine Learning

The most important functional requirement is to distinguish a likely baby-crying event from non-crying sounds. The STM32F407 has limited RAM for sustained audio buffering and machine-learning inference. Before selecting on-device inference, determine the actual memory footprint, feature-extraction costs, execution time, and impact on FreeRTOS deadlines. A backend classifier is permitted when justified by measured constraints.

Your classifier should satisfy the following baseline requirements:

- Acquire audio through an external microphone at a documented sample rate, duration, and format. Explain buffering and noise-handling choices.
- Implement a classifier with at least two labels: baby crying and non-crying/background sound. A publicly available model may be adapted, provided it is cited and evaluated on independent examples.
- Keep training/validation/test examples separate. Report the source, licensing, class balance, sample counts, and conditions of the test audio.
- Apply a justified confidence threshold and event confirmation rule (such as consecutive positive windows) to limit repeated or spurious notifications.
- Either deploy a demonstrably feasible lightweight model on the STM32F407 or send audio-derived features/short bounded windows to an authorized service for inference. Quantify RAM use, timing, throughput, latency, and privacy trade-offs.
- Do not store or stream continuous identifiable audio by default. Where audio must be transmitted for inference, use secure access and define how recordings are deleted.

Be careful when evaluating the classifier: playing the same audio used to train or select the model is not adequate evidence of generalization. Test unseen cries and common confusers such as speech, music, household noise, and silence.

Device Firmware

Once individual sensors, audio acquisition, and communication are working, integrate them in an STM32Cube HAL application scheduled by FreeRTOS. Use FreeRTOS tasks, queues, semaphores, mutexes, and software timers as appropriate. All hardware access must use verified STM32F4 HAL/LL drivers or carefully ported peripheral drivers. Your firmware should roughly follow this sequence:

- Initialize the board, logger, watchdog as appropriate, and onboard device drivers.
- Initialize the microphone interface and allocate audio buffers within the measured RAM budget.
- Create separate periodic or event-driven FreeRTOS tasks for sensing, audio acquisition/feature generation, connectivity, local display/alarm, and event logging.
- Connect the onboard Wi-Fi interface and the configured broker or web endpoint.
- Acquire readings, timestamp data, evaluate alert conditions, and publish well-defined messages without blocking critical acquisition tasks.
- Indicate cry and environmental alarms locally; accept a button acknowledgement and avoid uncontrolled repeated alarms.
- Detect Wi-Fi loss, queue essential event metadata locally where practical, attempt reconnection, and expose a device-fault state.

Use FreeRTOS queues, semaphores, task notifications, or mutexes when tasks exchange or share data. Demonstrate correct task priorities, nonblocking behavior for time-sensitive acquisition, avoidance of race conditions, and an explanation of any interrupt-to-task coordination. Measure heap consumption, minimum

remaining stack (high-water marks), task periods, and worst observed task latency. Log failures and implement sensible responses to sensor, Wi-Fi, and storage faults.

Backend Service and Connectivity

Your backend may run on a local workstation, a single-board computer, or a cloud instance. It should provide the following functionality:

- An MQTT broker or HTTP endpoint that receives data from RT-Spark; use authentication and appropriate transport protection.
- An audio inference endpoint or service when classification is not executed on the device.
- A database containing timestamped sensor readings, classified events, device status, and alert acknowledgements.
- A web API or equivalent service for the mobile application to retrieve recent and historical data.
- A notification mechanism that reports new cry events and meaningful environmental or device faults, with debouncing to prevent alert flooding.

Mosquitto, Flask/FastAPI, Node.js, PostgreSQL, or SQLite are possible technologies, but no particular product is mandatory. The onboard Wi-Fi module must be driven using an STM32 HAL-compatible interface and a networking implementation selected for this project; RT-Thread Wi-Fi packages cannot be used unchanged. Verify supported commands, TCP/IP capabilities, and transport security before selecting MQTT or HTTPS. Do not commit credentials or secrets to GitHub.

Mobile App

You should develop an Android application that displays and manages information from the baby monitoring prototype. The application should, at a minimum:

- Display live or recently updated room temperature, humidity, ambient light, cry-detection state, and device/network condition.
- Generate or receive a clearly labeled notification for a detected crying event.
- Show a searchable or chronological history of events and relevant sensor readings.
- Display the time of the last successful device update and distinguish stale values from current data.
- Provide an understandable way to acknowledge a notification and view local alert settings.
- Handle API errors and connectivity interruptions without freezing the interface.

A clear and accessible interface is more important than decorative effects. Choose appropriate labels, alert priority, readable contrast, and informative error messages. Do not present this classroom prototype as a medical or safety-certified monitoring device.

Testing and Evaluation

Your prototype must be demonstrated using controlled, non-hazardous test scenarios. Do not rely only on screenshots showing that the app runs.

Test area	Minimum evidence
Audio classification	At least 20 previously unseen crying and 20 non-crying test clips or episodes. Include a confusion matrix, precision, recall, F1-score, false positives, and failure examples.
End-to-end notification	At least 10 triggered-event trials. Record capture-to-notification time and successful/failed deliveries.
Onboard sensing	Show live, timestamped temperature/humidity, light, proximity, and device-motion/tilt readings with relevant controlled changes.

Test area	Minimum evidence
Real-time firmware	Demonstrate FreeRTOS concurrent tasks; record sampling periods, scheduling behavior, heap usage, and available stack margins.
Connectivity and resilience	Disconnect and restore Wi-Fi; show device status, reconnect behavior, local alerts, and post-recovery event handling.
User interface and controls	Demonstrate history, stale-data indication, acknowledgement, reset/restart, and normal/alert/fault states.

A reference sensor or instrument should be used to check ambient measurements where available; describe the conditions and limitations of any comparison. For the classifier, accuracy alone is insufficient because class imbalance and false alarms can conceal performance problems. Report measured results, explain limitations, and avoid unsupported performance claims.

Troubleshooting and Safety

You may encounter problems with firmware integration, audio buffers, network access, sensor timing, and notification delivery. Address them systematically:

- Verify one driver or package at a time before incorporating it into the full system.
- Check the actual RT-Spark board revision, schematic, pin assignments, clocks, power requirements, peripheral buses, and STM32Cube HAL configuration.
- Investigate allocation failures, dropped audio windows, lost messages, race conditions, and task starvation using logs and measured resource use.
- Treat an unreachable server, missing sensor, or exhausted queue as a defined fault, not as proof that the monitored room is safe.
- Demonstrate using a classroom test fixture and prerecorded, licensed audio. Do not attach electronics, cables, or components to an infant or place them within an infant sleep space.

The device is an educational prototype only and cannot replace direct adult supervision or a certified infant-monitoring or medical device. Do not claim the proximity sensor detects breathing, that a stationary device detects infant movement, or that a cry classifier diagnoses a medical condition.

Project Deliverables

Submit the following through the designated course channel:

- A GitHub repository containing the complete PlatformIO project (platformio.ini and source files), STM32Cube HAL + FreeRTOS firmware, backend/classifier, mobile app, README, wiring, dependency versions, build/flash instructions, and a tagged submission version.
- A project report containing the objective, architecture, component inventory, external-module justification, wiring/pin diagram, task/communication design, and data flow.
- Machine-learning documentation: dataset provenance, model approach, training/test protocol, confusion matrix, evaluation results, and inference location rationale.
- Test results: unit tests for event logic/data handling, static-analysis findings, logs, numerical alert-latency and classification results, memory/stack measurements, and fault-recovery evidence.
- A short demonstration video showing the live device, app notifications, event history, controls, and at least one network failure/recovery test.

Every team member should be able to explain the subsystem they implemented and how it exchanges information with the other components. Code taken or adapted from public examples must be attributed. A copied demonstration without system-level adaptation and independent validation will not satisfy this project.

Assessment Criteria (100 Points)

Criterion	Points	What will be assessed
System design and component justification	15	Valid architecture; full onboard inventory; external-module justification; pinout, board manifest and interface design.
STM32Cube HAL + FreeRTOS firmware	25	Working board drivers; correctly scheduled FreeRTOS tasks; synchronization; local status/controls; measured memory and timing.
Baby-cry ML pipeline	20	Functional pipeline; independently tested classifier; appropriate labels and thresholds; reasoned deployment.
Backend, connectivity, and mobile app	20	End-to-end data flow; secure handling; timely alerts; history; robust error and stale-data states.
Testing, reliability, and safety	15	Controlled functional tests; quantitative metrics; fault recovery; documented safety/privacy limitations.
Documentation and reproducibility	5	Reproducible PlatformIO build/upload; GitHub history; README, report, tests, and evidence.

A working demonstration is required. Additional features such as voice-assistant integration, over-the-air updates, or advanced visualization are optional improvements; they do not substitute for missing core requirements.